

FINAL PROJECT REPORT
IF670 CROSS-PLATFORM MOBILE PLATFORM

GROUP 8: SIGAP



Meshach Christian	0000068876
Richie Christian Tungadi	0000074763
Damar Nur Rafli	0000074723
Meylani Alfatikha	0000098358
Juanto	0000091554

INFORMATICS STUDY PROGRAM
FACULTY OF ENGINEERING AND INFORMATICS
UNIVERSITAS MULTIMEDIA NUSANTARA
2026

ABSTRACT

Natural disasters may happen anytime and anywhere, such as floods, earthquakes, landslides, and extreme weather. In many cases, people do not receive information quickly enough, causing delayed evacuation and increasing the risk of casualties and property damage. In addition, many people still lack education regarding disaster preparedness and mitigation procedures, resulting in confusion during emergency situations. This project proposes a crowdsourcing-based disaster management mobile application integrated with the Meteorology, Climatology, and Geophysical Agency API to provide real-time disaster information and public reporting features. The application allows users to submit disaster reports, view disaster locations through interactive maps, and receive real-time earthquake data fetched from the BMKG API, displayed directly on the home screen to keep users informed of the latest seismic activity regarding earthquakes, weather warnings, and other emergency alerts. Furthermore, the application provides disaster mitigation education to improve public awareness and preparedness. The proposed system is expected to help communities respond more quickly and effectively during disaster situations while improving coordination and access to accurate information.

Keywords: Natural disaster, Crowdsourcing application, BMKG API, Push notification, Disaster mitigation

TABLE OF CONTENTS

ABSTRACT.....	2
CHAPTER I	
INTRODUCTION.....	8
1.1 Background.....	8
1.1.1 Review of Previous Research.....	9
1.1.2 Comparative Analysis of Similar Applications.....	10
1.2 Problem Formulation.....	12
1.3 Problem Limitations.....	12
1.4 Purpose and Benefits.....	13
CHAPTER II	
APP DEVELOPMENT.....	15
2.1 Android Project Structure.....	15
2.2 Database Structure.....	22
2.3 Flowchart.....	24
2.3.1 Application Flowchart.....	25
2.3.1 Wireframe Design.....	25
2.4 Native Device Features Implementation.....	29
2.4.1 Device Features Covered in Lectures.....	29
2.4.2 Device Features Beyond Lectures.....	30
2.5 Internal Code Documentation.....	31
2.5.1 Documentation Standards.....	31
2.5.2 Documentation Examples.....	31
CHAPTER III	
RESULT & DISCUSSION.....	34
3.1 Application Flow.....	34
3.2 Application of 8 Golden Rules of Interface Design.....	36
3.3 Final Result.....	38
Figure 1.4 — SIGAP Incidents List Screen.....	41
Figure 1.5 — SIGAP Incident Cards.....	42

Figure 1.6 — SIGAP Crisis Map Screen.....	43
Figure 1.7 — SIGAP Profile Screen.....	44
Figure 1.8 — SIGAP Report Incident Screen.....	45
Figure 1.9 — SIGAP Report Incident — Impact & Evidence.....	46
Figure 1.10 — SIGAP BMKG Earthquake Screen.....	47
Figure 1.11 — SIGAP Verification / Condition Update Modal.....	48
3.4 Discussion.....	49
3.4.1 User Authentication.....	49
3.4.2 Incident Reporting.....	49
3.4.3 Crisis Map.....	50
3.4.4 BMKG Earthquake Alert Integration.....	50
3.4.5 Community Verification and Trust System.....	51
3.4.6 Push Notifications.....	51
3.4.7 SOS Feature and Gyroscope-Based Detection.....	52
3.4.8 Admin Moderation Panel.....	52
3.4.9 Overall Evaluation.....	53
3.5 Testing and Evaluation.....	53
3.5.1 Black Box Testing.....	53
3.5.2 Test Documentation — Errors and Bugs.....	55
3.5.3 User Evaluation and Response to Feedback.....	56
CHAPTER IV	
CONCLUSION.....	58
4.1 Conclusion.....	58
4.2 Suggestion.....	58
ATTACHMENT GROUP MEMBER’S PERSONAL DATA.....	62

CHAPTER I

INTRODUCTION

1.1 Background

Indonesia is one of the countries that is highly vulnerable to disasters and emergency situations, such as floods, earthquakes, landslides, fires, and traffic accidents. These incidents often cause injuries, fatalities, infrastructure damage, and public disruption. In particular, traffic accidents and natural disasters remain major public safety concerns that require fast response and accurate information delivery.

Indonesia ranks among the world's most disaster-prone nations. According to the National Disaster Management Authority (BNPB), Indonesia recorded over 5,400 disaster events in 2023 alone, resulting in more than 600 fatalities and affecting over 5.5 million people. Floods remain the most frequent disaster type, accounting for approximately 34% of all events, followed by landslides (18%) and extreme weather (16%). The 2024 BNPB annual report further highlights that disaster response times remain critically slow in rural and semi-urban areas, with an average emergency response lag of 2–4 hours due to fragmented communication and the absence of integrated reporting platforms.

In the digital era, smartphone penetration in Indonesia has reached over 73% of the population as of 2024 (Statista, 2024), creating a strong foundation for mobile-based community reporting systems. Despite this potential, no single platform currently unifies official government disaster alerts with real-time community incident reporting in a cross-platform mobile application accessible to the general public.

In many situations, incidents are not reported promptly, resulting in delays in emergency response and assistance for victims. In addition, people near the incident location often

do not have access to accurate and real-time information regarding nearby emergencies. This condition can reduce public preparedness and slow down coordination between communities and emergency responders.

Currently, several applications such as Google Maps and Waze provide traffic monitoring and incident reporting features. These applications are useful for navigation and road condition monitoring; however, they mainly focus on traffic-related information and do not integrate official disaster alerts with community-based incident reporting in a single platform.

In Indonesia, BMKG provides official disaster and weather warnings through public alert systems and APIs. However, official alerts and public incident reports are usually separated across different platforms, making it difficult for users to access complete emergency information efficiently.

With the advancement of mobile and digital technology, applications can be utilized to provide real-time incident reporting, location-based notifications, and official emergency alerts. Through the integration of BMKG Alert API, users can receive official disaster warnings while also reporting incidents occurring around them. This integration can improve public awareness, information accessibility, and emergency response efficiency.

Therefore, the “Incident Reporting Application with BMKG Alert API Integration” was developed to help communities report incidents faster, receive official emergency alerts, and improve communication during emergency situations. This application is expected to become a useful platform for creating a safer, more informed, and better-prepared society in responding to disasters and public incidents.

1.1.1 Review of Previous Research

Several previous studies have explored the development of disaster reporting and emergency management applications using mobile technology. Razi & Shahdan (2014)

developed a mobile-based flood reporting system that allowed communities to submit real-time flood reports via smartphones, demonstrating the feasibility of crowdsourced disaster data collection. Their findings showed that community participation significantly improved the speed and accuracy of flood information dissemination.

Imran et al. (2015) conducted research on social media-based disaster information processing, highlighting the importance of real-time data aggregation and filtering during emergency events. Their study emphasized that integrating multiple data sources, including both official alerts and community reports, produces more comprehensive situational awareness compared to single-source systems.

Kurniawan et al. (2019) proposed a location-based disaster warning system for Indonesian communities integrated with BMKG seismic data, finding that automatic push notifications triggered by official API data significantly reduced response time among users. This research directly supports the approach adopted in SIGAP, which similarly integrates the BMKG Open Data API with community-based incident reporting.

Utomo et al. (2021) evaluated the usability of disaster management mobile applications in Indonesia, concluding that applications with clear navigation, real-time map visualization, and community verification features received significantly higher usability scores. These findings informed the design decisions in SIGAP, particularly the implementation of the crisis map and community trust scoring system.

Based on the review of previous research, existing systems either rely solely on official data sources or focus exclusively on community reports without integration of authoritative data. SIGAP addresses this gap by combining BMKG official earthquake alerts with community-submitted incident reports in a unified platform.

1.1.2 Comparative Analysis of Similar Applications

To identify the gaps that SIGAP aims to address, a comparative analysis was conducted against three existing applications with similar functionalities: Qlue, PetaJakarta.org, and the official BMKG application.

Criteria	SIGAP	Qlue	PetaJakarta.org	BMKG App
Platform	Android & iOS (React Native)	Android & iOS	Web-based	Android & iOS
Community Reporting	Yes	Yes	Yes	No
Official API Integration	Yes (BMKG)	No	Partial (Twitter)	Yes (BMKG)
Real-time Map	Yes	Yes	Yes	No
Community Verification	Yes (Trust Score)	No	No	No
Push Notifications	Yes (FCM)	Yes	No	Yes
Multilingual Support	Yes (EN & ID)	Indonesian only	Indonesian only	Indonesian only
Sensor Integration	Yes (Gyroscope)	No	No	No

Based on the comparative analysis, Qlue focuses primarily on urban complaint reporting and lacks integration with official disaster APIs. PetaJakarta.org is a web-based platform that aggregates social media flood reports but does not provide push notifications or a dedicated mobile application. The official BMKG application provides authoritative earthquake and weather data but does not support community incident reporting or real-time crisis maps.

SIGAP differentiates itself by combining official BMKG data integration, community-based incident reporting, real-time crisis mapping, community trust verification, and gyroscope-based sensor detection within a single cross-platform mobile application. This integrated approach addresses the limitations found in each existing

application and provides a more comprehensive emergency management solution for Indonesian communities.

1.2 Problem Formulation

Based on the background above, the problems discussed in this project are:

1. How can a mobile application provide fast and efficient real-time incident reporting?
2. How can BMKG Alert API be integrated to deliver official disaster and weather alerts effectively?
3. How can location-based emergency information improve public awareness and preparedness?
4. How can technology support faster communication and response during emergency situations?

1.3 Problem Limitations

The limitations of this project are as follows:

1. The application only focuses on incident reporting and BMKG alert notification features.
2. The application is designed for mobile platform users with internet access.
3. The reporting system is limited to user-generated reports and location-based information.
4. Disaster and weather alert information depends on data provided by the BMKG Alert API.
5. The application only supports real-time notification and information delivery related to emergency situations.

1.4 Purpose and Benefits

The objectives of this project are:

1. To develop a mobile application that enables users to report incidents in real time.
2. To integrate BMKG Alert API for delivering official disaster and weather alerts to users.
3. To provide accurate and accessible location-based emergency information.
4. To support faster communication and coordination during emergency situations.
5. To utilize digital technology to improve public awareness and emergency preparedness.

The benefits of this project are as follows:

1. For the community, this application helps users obtain incident and emergency information quickly and accurately.
2. For emergency response efforts, the application supports faster communication and coordination through real-time reporting and alert notifications.
3. For users, the application increases awareness regarding emergency situations and public safety.
4. For future researchers, this project can serve as a reference for the development of similar applications related to emergency reporting and alert systems.
5. For technological development, this project encourages the implementation of digital solutions to improve community preparedness and public safety.

1.5 User Definition

SIGAP is designed to serve three primary user personas:

Persona 1 — General Public User (Warga Umum) Name: Budi, 32, warehouse worker in Tangerang Goals: Quickly report incidents near his area (floods, road blockages) and receive nearby emergency alerts without needing to understand complex government systems. Pain Points: Does not follow official BMKG channels; relies on WhatsApp groups for disaster info which is often inaccurate or delayed. How SIGAP helps: Simple reporting form, one-tap location detection, real-time push notifications for nearby incidents.

Persona 2 — Community Volunteer / Relawan Name: Sari, 26, active in a local disaster volunteer group (KSB) Goals: Monitor multiple incident reports across her area, verify accuracy of reports, and coordinate community response. Pain Points: Existing tools (Qlue, WhatsApp) lack a trust/verification system — she can't tell which reports are real. How SIGAP helps: Community verification system with trust scores, crisis map with category filters, incident thread for coordination.

Persona 3 — Administrator / Moderator Name: Admin team, Universitas Multimedia Nusantara Goals: Moderate reported content, remove false or harmful reports, and maintain platform integrity. Pain Points: No existing platform gives moderators a dedicated mobile panel for real-time moderation. How SIGAP helps: Role-based access control, dedicated moderation queue panel accessible directly from the app.

CHAPTER II

APP DEVELOPMENT

2.1 Android Project Structure

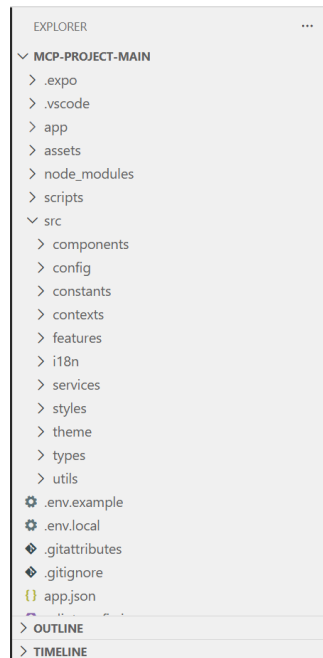


Figure 2.1 — SIGAP Android Project Structure

The SIGAP application is built using Expo with file-based routing provided by Expo Router. The project follows a feature-based architecture to separate concerns and maintain scalability. The overall project structure is organized as follows:

app/ — Contains all route files managed by Expo Router. Routes are kept thin with no business logic.

(tabs)/ — Tab-based navigation screens: index.tsx (Home), map.tsx, report.tsx, incidents.tsx, analytics.tsx, education.tsx, profile.tsx, detail.tsx

- **_layout.tsx** — Root navigation layout and route guards

- **login.tsx** — Login screen
- **register.tsx** — Registration screen
- **moderation.tsx** — Admin moderation panel

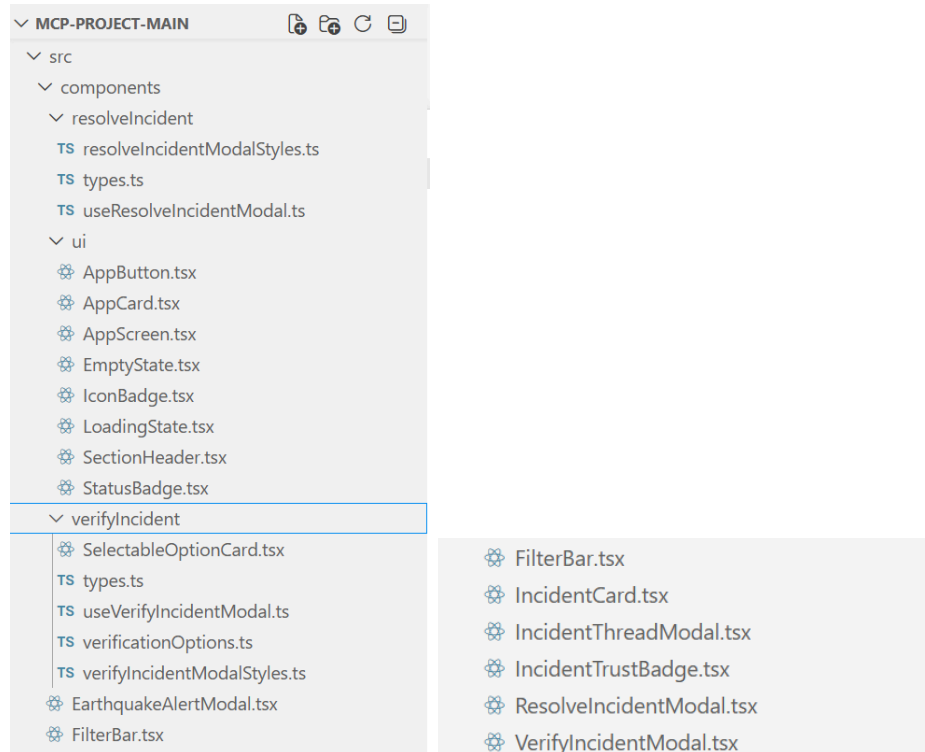


Figure 2.2 — screenshot `src/components`

src/components/ — Contains shared UI components including `AppButton`, `AppCard`, `AppScreen`, `EmptyState`, `IconBadge`, `LoadingState`, `SectionHeader`, `StatusBadge`, `resolveIncident`, `verifyIncident`, `FilterBar`, `IncidentCard`, `IncidentThreadModal`, and `IncidentTrustBadge`.

src/constants/ — Domain constants including `categoryMeta`, `distances`, `helpers`, `options`, `typeMeta`, `cloudinaryConfig`, `incident`, and `reportTaxonomy`.

src/context/ — `AuthContext.tsx` for managing user authentication state.

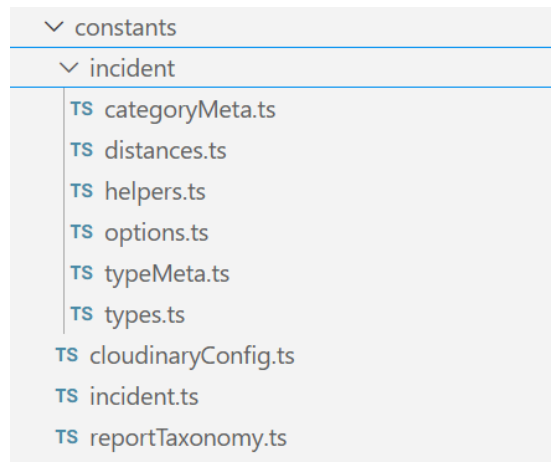


Figure 2.3 — SIGAP src/constants Structure

src/constants/ — Contains domain constants and configuration values organized into the following files:

- **incident/** — A subdirectory containing incident-specific constants:
 - **categoryMeta.ts** — Metadata for each incident category
 - **distances.ts** — Distance threshold constants used for proximity calculations
 - **helpers.ts** — Helper functions for incident constants
 - **options.ts** — Selectable options for incident forms
 - **typeMeta.ts** — Metadata for each incident type
 - **types.ts** — TypeScript type definitions for incident constants
- **cloudinaryConfig.ts** — Cloudinary upload preset and cloud name configuration
- **incident.ts** — Core incident constants used across the application
- **reportTaxonomy.ts** — Report category and subcategory taxonomy definitions

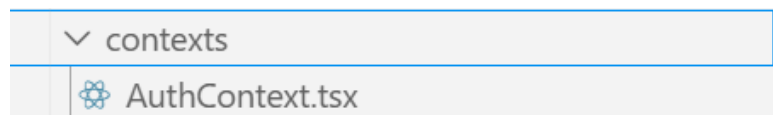


Figure 2.4 — SIGAP src/contexts Structure

src/contexts/ — Contains React context providers used for global state management across the application.

- **AuthContext.tsx** — Manages user authentication state including login, logout, registration, and session persistence. This context is wrapped around the entire application to ensure authentication status is accessible from any screen.

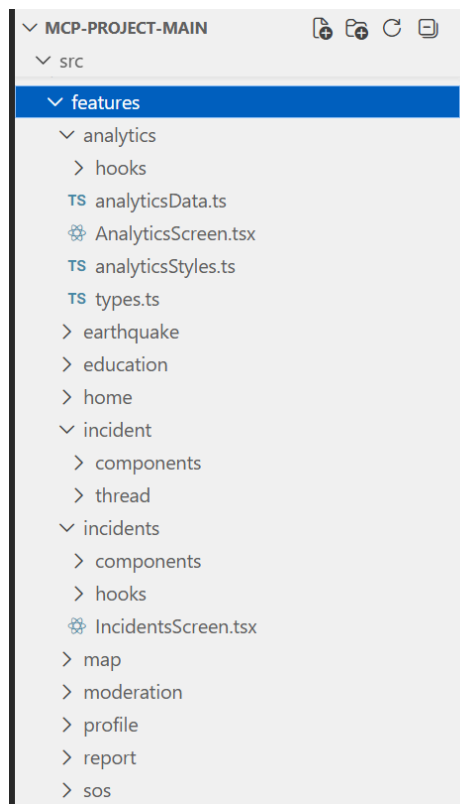


Figure 2.5 — SIGAP src/features Structure

src/features/ — Contains feature-specific screens, hooks, components, and utilities organized by domain:

- **analytics/** — Incident insights and statistics dashboard, including hooks/, analyticsData.ts, AnalyticsScreen.tsx, analyticsStyles.ts, and types.ts
- **earthquake/** — BMKG earthquake alert screen and related components
- **education/** — Disaster safety guidelines and education content

- **home/** — Home screen and live monitoring dashboard
- **incident/** — Incident detail view organized into:
 - **components/** — UI components for incident display
 - **thread/** — Incident thread, discussion, and reply components
- **incidents/** — Incident list screen organized into:
 - **components/** — Reusable incident list components
 - **hooks/** — Custom hooks for data fetching and filtering
 - **IncidentsScreen.tsx** — Main incidents list screen
- **map/** — Crisis map with incident pins and category filters
- **moderation/** — Admin moderation panel for reviewing flagged reports
- **profile/** — User account information and contribution summary
- **report/** — Incident reporting form and submission logic
- **sos/** — SOS information and emergency contacts

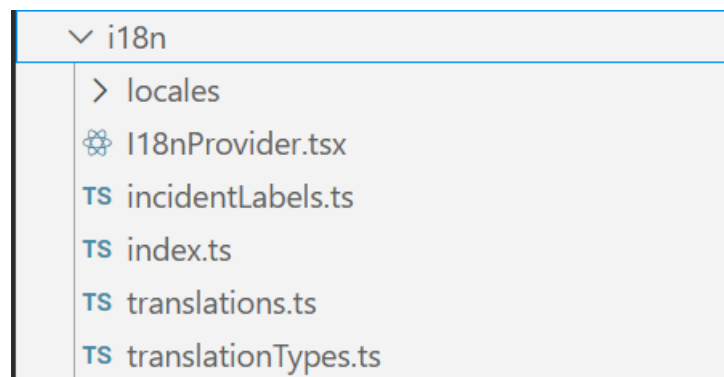


Figure 2.6 — SIGAP src/i18n Structure

src/i18n/ — Contains internationalization configuration supporting both English and Indonesian languages:

- **locales/** — Language locale files for each supported language
- **I18nProvider.tsx** — React provider component that wraps the application and supplies translation context to all screens
- **incidentLabels.ts** — Translated labels specific to incident categories and types
- **index.ts** — Entry point that exports the i18n hook and utilities

- **translations.ts** — Main translation strings for both English and Indonesian
- **translationTypes.ts** — TypeScript type definitions for translation keys to ensure type-safe translations throughout the application

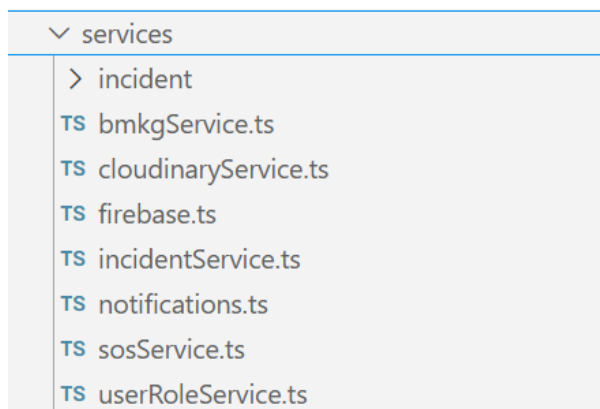


Figure 2.7 — SIGAP *src/services* Structure

src/services/ — Contains external service integrations and data persistence logic:

- **incident/** — Incident-specific service logic and operations
- **bmkgService.ts** — Integration with the official BMKG earthquake API for fetching real-time seismic data
- **cloudinaryService.ts** — Image upload and management via Cloudinary
- **firebase.ts** — Firebase initialization, configuration, and connection setup
- **incidentService.ts** — Firestore database operations for creating, reading, and updating incident reports
- **notifications.ts** — Push notification handling and scheduling
- **sosService.ts** — SOS alert service for emergency signal broadcasting
- **userRoleService.ts** — User role and permission management for differentiating regular users from moderators and admins

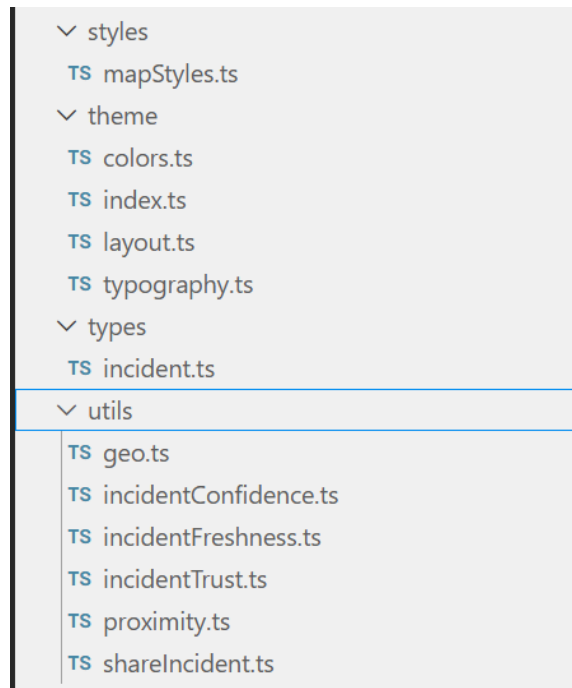


Figure 2.8 — SIGAP `src/styles`, `theme`, `types`, and `utils` Structure

src/styles/ — Contains shared StyleSheet modules used across multiple screens:

- **mapStyles.ts** — Style definitions for the crisis map screen

src/theme/ — Contains design tokens to maintain visual consistency across all screens:

- **colors.ts** — Color palette including primary, danger, success, warning, and neutral colors
- **index.ts** — Entry point that exports all theme tokens
- **layout.ts** — Spacing, padding, border radius, and shadow definitions
- **typography.ts** — Font sizes, font weights, and text style definitions

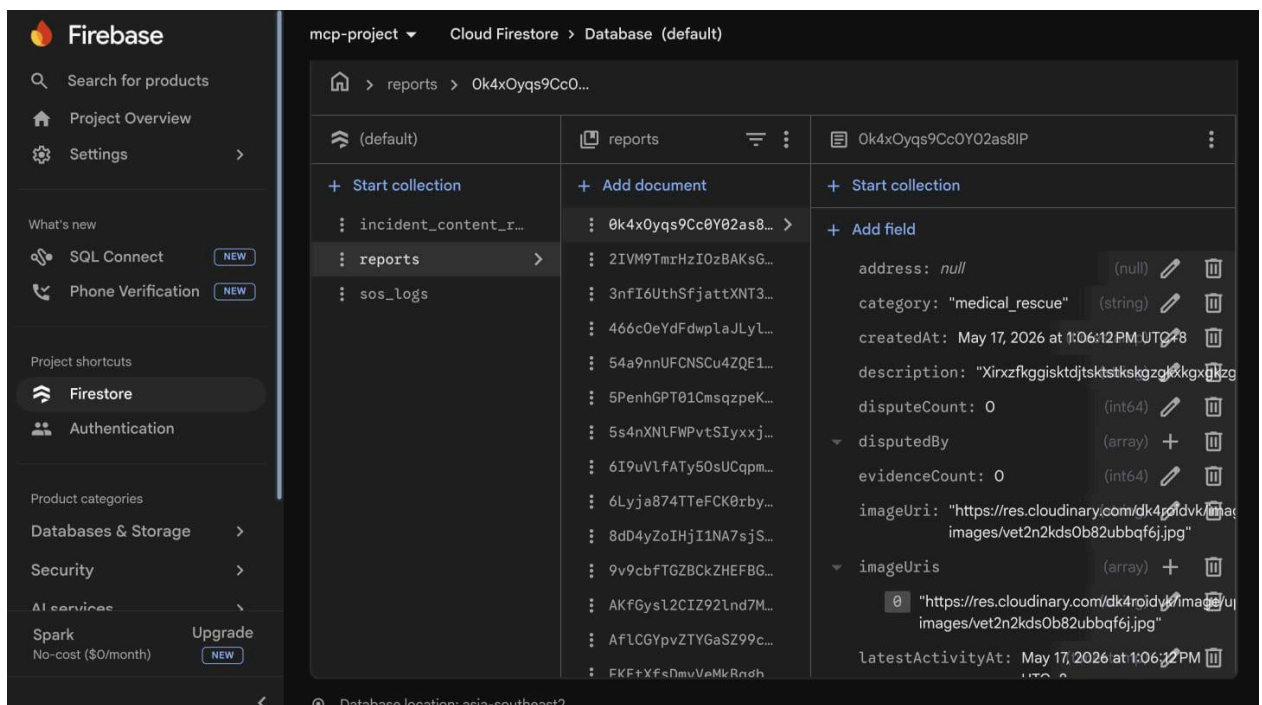
src/types/ — Contains shared TypeScript type definitions for core domain entities:

- **incident.ts** — Type definitions for IncidentReport, IncidentCategory, IncidentSeverity, and related data models

src/utils/ — Contains shared pure utility functions:

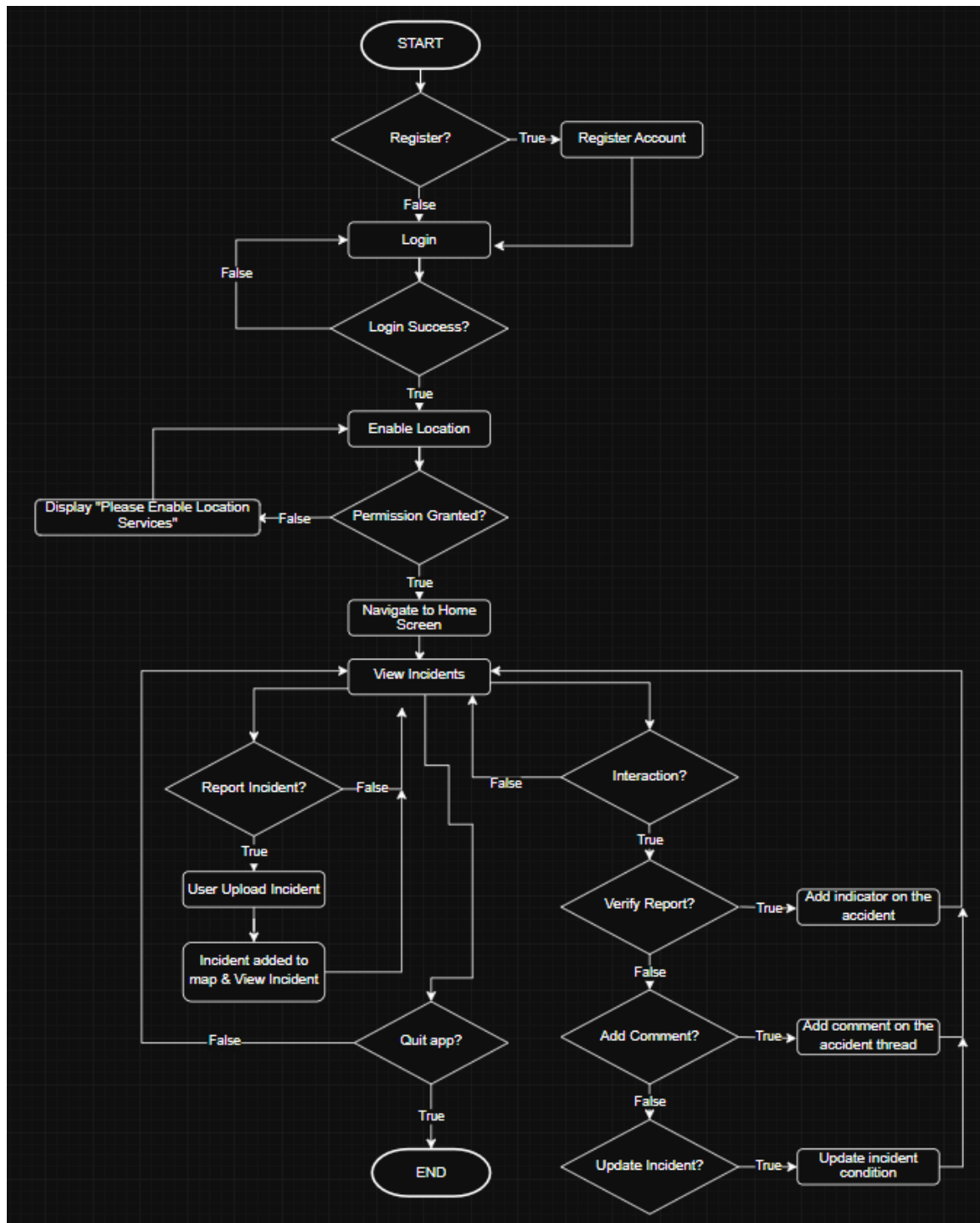
- **geo.ts** — Geolocation calculation utilities
- **incidentConfidence.ts** — Confidence score calculation for incident reports
- **incidentFreshness.ts** — Freshness score based on incident timestamp
- **incidentTrust.ts** — Trust level calculation based on community verification
- **proximity.ts** — Distance and proximity filtering utilities
- **shareIncident.ts** — Social media sharing functionality for incident reports

2.2 Database Structure



This is the Cloud Firestore database structure used in the SIGAP application. The database stores incident report data submitted by users in real time. Each report document contains important information such as incident category, description, image URL, timestamp, dispute count, evidence count, and activity status. The system uses Firebase Firestore to support real-time data synchronization, allowing incident information to be updated and accessed quickly by all users within the application.

2.3 Flowchart



2.3.1 Application Flowchart

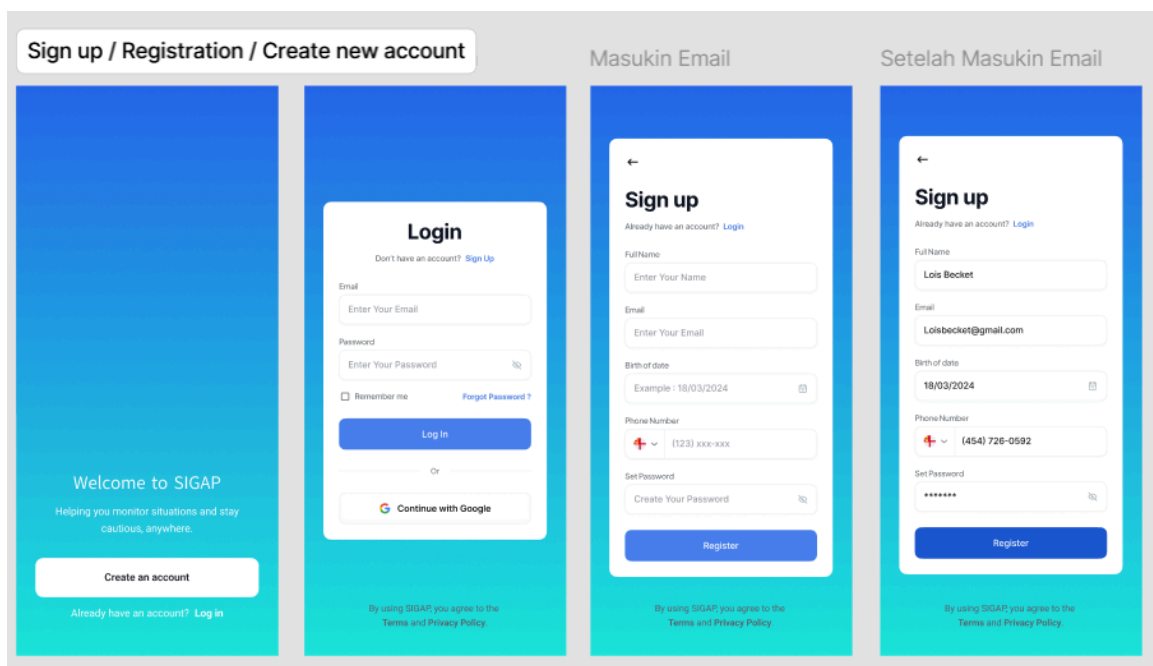
The application flowchart (Figure 3.1) illustrates the complete user journey from initial launch through all core features. The flow begins with user authentication (register or login), proceeds through location permission granting, and then branches into the main feature set: incident viewing, reporting, map exploration, BMKG data access, and community interaction (verify, comment, update). Admin users have an additional branch to the moderation panel based on role-based access control.

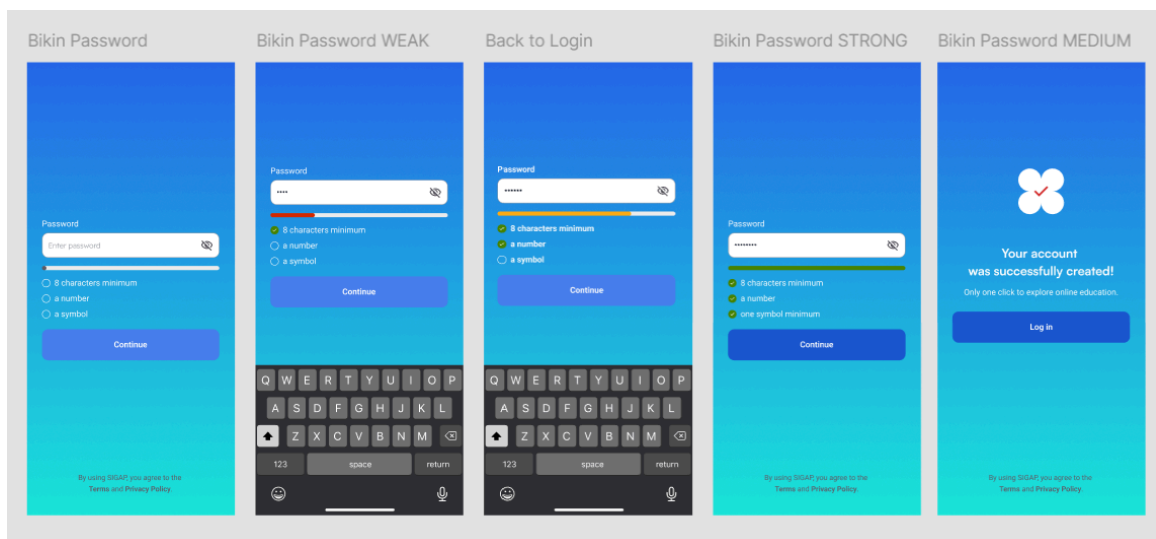
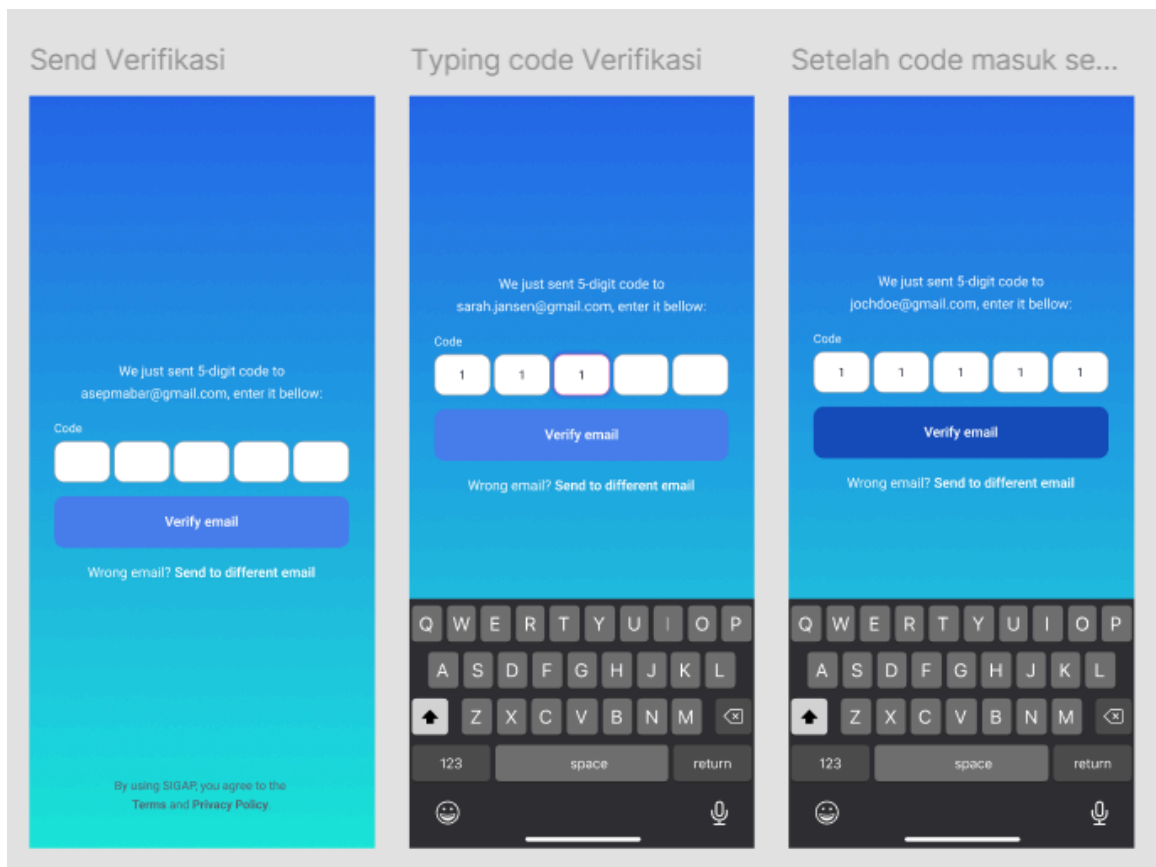
2.3.1 Wireframe Design

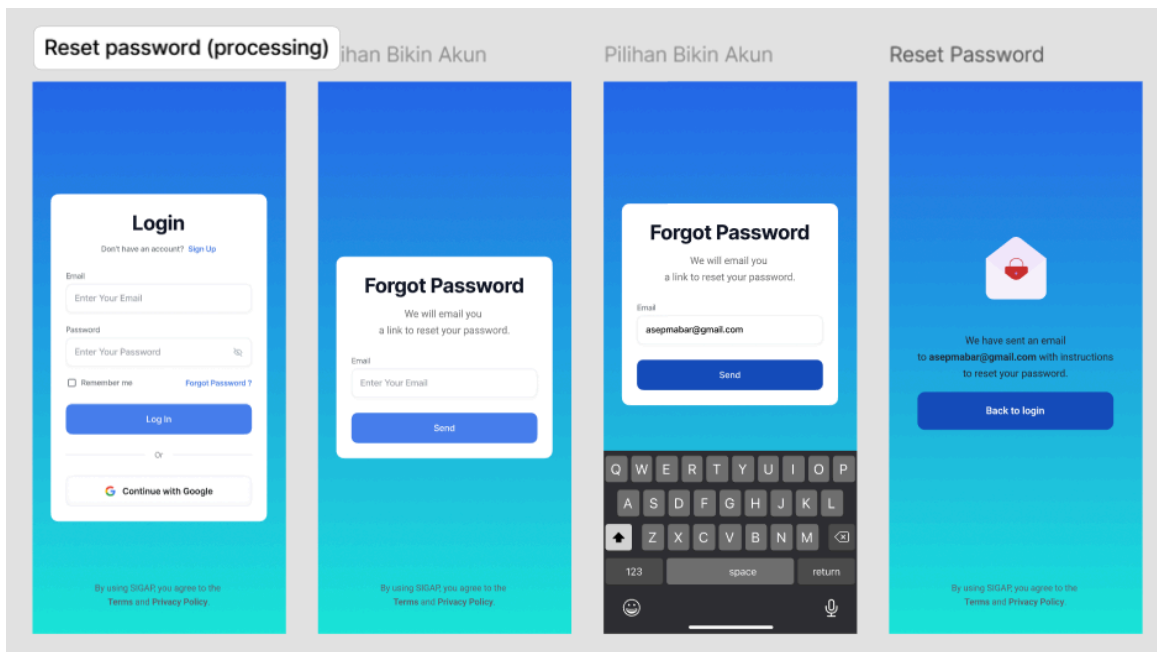
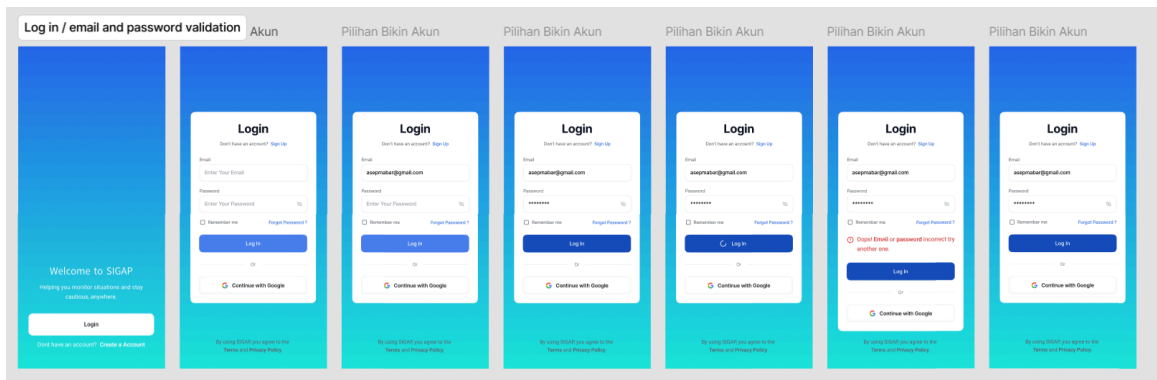
The SIGAP wireframe was designed in Figma prior to development and served as the primary UI reference throughout the build process. The wireframe covers the following screens:

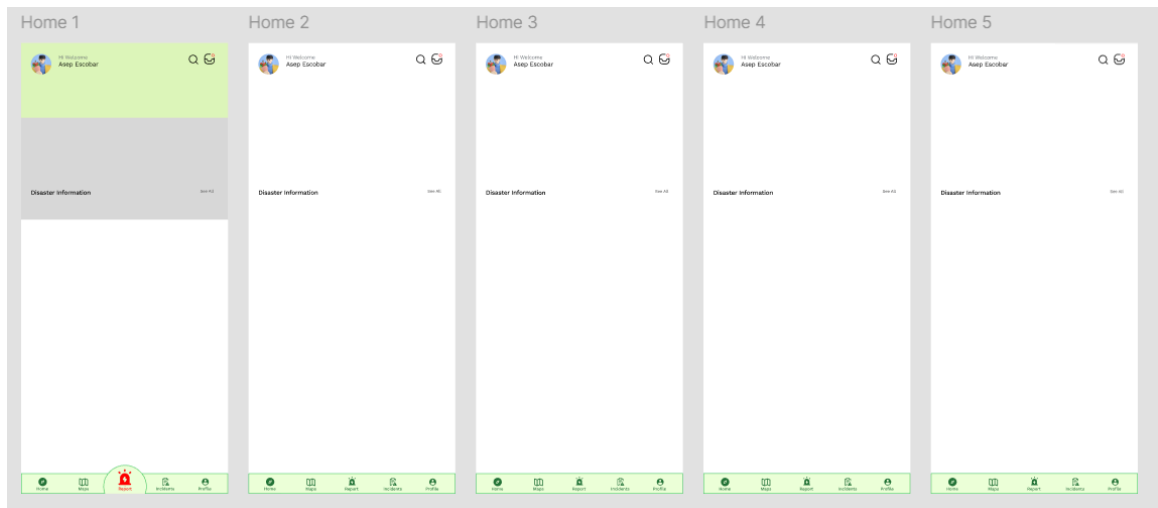
- **Login & Register Screen** — Email/password input fields, sign-in button, and navigation link to registration. The branding uses the SIGAP red (#C00000) as the primary background color to immediately establish visual identity.
- **Home Screen** — Displays a live monitoring summary (active incident count, high-urgency count), quick-access buttons to Report, Map, Analytics, BMKG, and Education features, and a scrollable BMKG Earthquake Updates card at the bottom.
- **Incidents List Screen** — Shows a real-time list of all community incidents with status filter tabs (All, Active, Resolved, High Urgency), a search bar, and summary stats cards at the top. Each incident card shows category icon, confidence score badge, urgency level, status, and timestamp.
- **Crisis Map Screen** — Interactive Google Maps view displaying incident pins color-coded by category. Includes filter chips (All, Active, Resolved, Category) and a floating Report button. Nearest incident is shown at the bottom of the screen.

- **Report Incident Screen** — A multi-step form: (1) location selection via current GPS or manual pin adjustment, (2) incident category selection, (3) impact assessment toggles, (4) optional context/description, and (5) photo evidence upload via camera or gallery.
- **BMKG Earthquake Screen** — Dedicated screen showing the latest significant earthquake (M5.0+) with full detail (magnitude, location, depth, coordinates, tsunami potential) and a secondary list of recent earthquakes.
- **Profile Screen** — Displays user account information, personal contribution statistics (total/active/resolved/high-urgency reports), account settings, and moderator panel access (role-dependent). Includes logout button.
- **Moderation Panel** — Admin-only screen showing a queue of flagged community reports with options to hide, approve, or escalate content.









2.4 Native Device Features Implementation

2.4.1 Device Features Covered in Lectures

The following device-specific features implemented in SIGAP correspond to topics discussed during lectures:

- **Geolocation** — The application uses the device's GPS to detect the user's current location for accurate incident pinning on the crisis map. Implemented via expo-location in the map and report features.
- **Camera & Gallery** — Users can attach photo evidence when submitting incident reports. The application accesses the device camera (expo-camera) and image gallery (expo-image-picker) to capture and upload images to Cloudinary.
- **Push Notifications** — Firebase Cloud Messaging (FCM) is integrated to deliver real-time push notifications to users when new incidents are reported nearby or when a significant earthquake is detected. Implemented in `src/services/notifications.ts`.

- **Local Storage** — AsyncStorage is used to persist user session and authentication state locally on the device, enabling seamless re-login without re-authentication.

2.4.2 Device Features Beyond Lectures

In addition to the features covered in lectures, SIGAP implements several advanced device-specific capabilities that go beyond the standard course material:

- **Gyroscope / Motion Sensor** — The application integrates the device's accelerometer and gyroscope sensor via expo-sensors to detect abnormal motion patterns that may indicate a landslide event. When unusual vibration thresholds are exceeded, the system triggers an early warning alert to the user. This feature is implemented in the sos/ feature module and serves as an innovative hardware-based disaster detection mechanism.
- **SOS Emergency Signal Broadcasting** — The sosService.ts module implements an SOS alert broadcast feature that leverages multiple device capabilities simultaneously: GPS for location, network for data transmission, and notification channels for immediate alerts. This multi-sensor coordination goes beyond standard single-feature implementations covered in lectures.
- **Real-Time Seismic Data Integration (BMKG API)** — The bmkgService.ts integrates with the official BMKG Open Data API to fetch live earthquake data. The application applies a magnitude threshold filter (M5.0+) and automatically displays an EarthquakeAlertModal on the home screen, providing real-time hazard awareness without user interaction. This represents a beyond-lecture integration of an external government API with native UI triggers.
- **Community Verification Trust System** — The application computes a dynamic trust score for each incident report using three utility functions:

`incidentConfidence.ts` (confidence based on verification count), `incidentFreshness.ts` (time-decay scoring), and `incidentTrust.ts` (combined trust level from community votes). This algorithmic approach to crowd-sourced verification is a novel implementation not covered in standard lectures.

- Incident Social Sharing — The `shareIncident.ts` utility enables users to share incident reports directly to social media platforms using the device's native Share API (expo-sharing), allowing emergency information to spread beyond the application ecosystem.

2.5 Internal Code Documentation

The SIGAP codebase follows a consistent internal documentation standard to ensure code readability, maintainability, and ease of collaboration among team members. All major functions, services, hooks, and components are documented using JSDoc-style comments in TypeScript.

2.5.1 Documentation Standards

The following documentation conventions are applied throughout the project:

- JSDoc comments (`@param`, `@returns`, `@description`) on all exported functions and hooks.
- Inline comments explaining non-obvious logic, threshold values, and algorithm steps.
- TypeScript type definitions serving as self-documenting interfaces for all data models.
- Descriptive variable and function naming following camelCase conventions.

2.5.2 Documentation Examples

The following code snippets illustrate the internal documentation practice applied in the SIGAP codebase:

Example 1 — bmkgService.ts (API Integration)

```
/**
 * Fetches the latest earthquake data from BMKG Open Data API.
 * Filters results to only return earthquakes with magnitude >= 5.0
 * to reduce noise and focus on significant seismic events.
 * @returns {Promise<EarthquakeData[]>} Array of significant
earthquake events
 * @throws {Error} If the BMKG API is unreachable or returns invalid
data
 */
export const fetchLatestEarthquakes = async ():
Promise<EarthquakeData[]> => {
  // Magnitude threshold: only alert for M5.0+ to avoid false alarms
  const MAGNITUDE_THRESHOLD = 5.0;
```

Example 2 — incidentTrust.ts (Trust Scoring Algorithm)

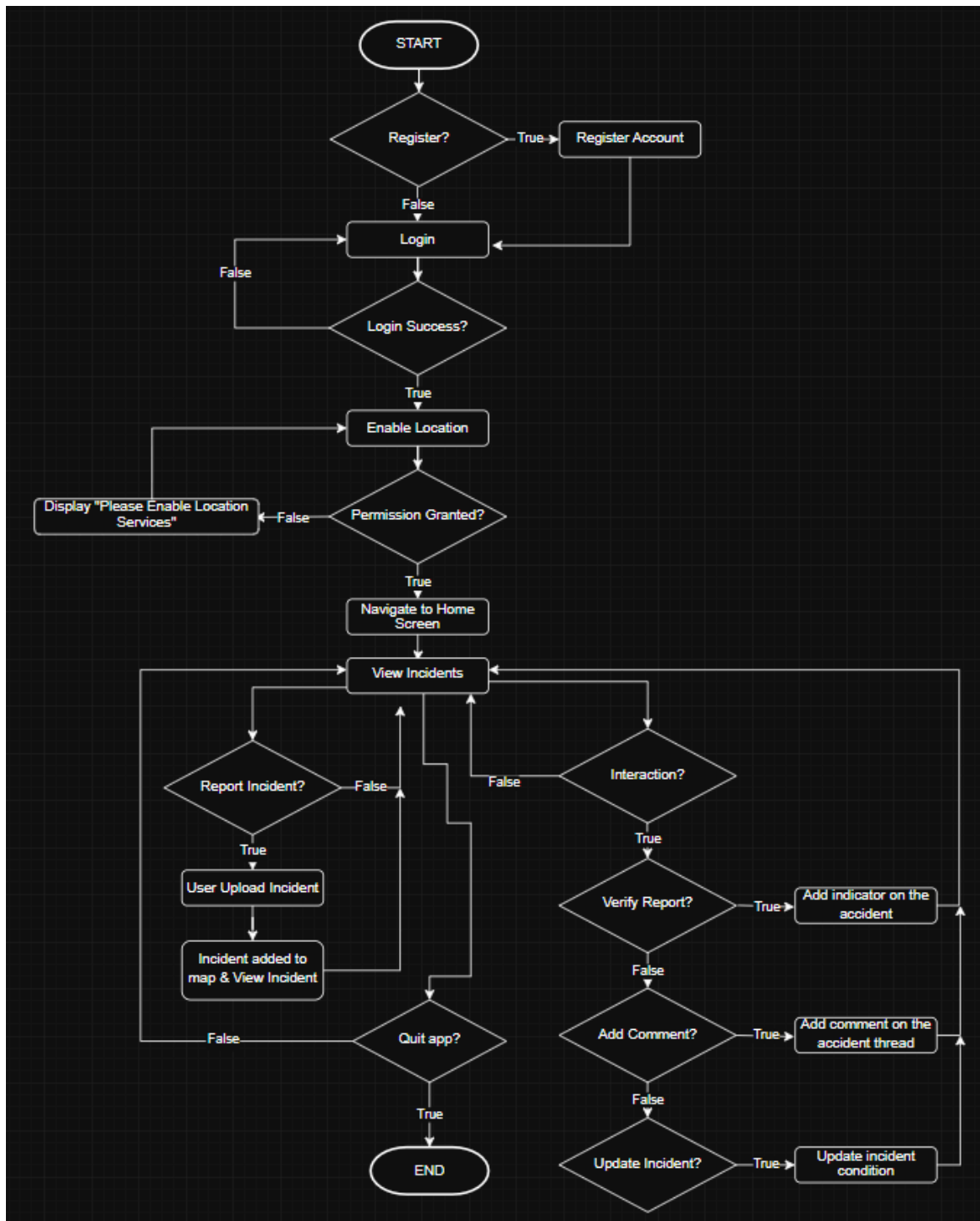
```
/**
 * Calculates a composite trust score for an incident report.
 * Combines confidence (verification count), freshness (time decay),
 * and dispute penalty into a normalized 0-100 score.
 * @param {IncidentReport} incident - The incident document from
Firestore
 * @returns {number} Trust score between 0 (untrusted) and 100 (fully
verified)
 */
export const calculateTrustScore = (incident: IncidentReport): number
=> {
  const confidence = getIncidentConfidence(incident); // 0-100
  const freshness = getIncidentFreshness(incident); // 0-1 decay
  // Weighted formula: 60% confidence + 40% freshness, penalized by
disputes
```

These documentation practices ensure that every team member can understand the purpose, inputs, and outputs of each module without needing to trace through implementation details. The consistent use of JSDoc across all service files, utility functions, and custom hooks significantly reduces onboarding time and supports long-term maintainability of the codebase.

CHAPTER III

RESULT & DISCUSSION

3.1 Application Flow



This application flow describes a community-based disaster and accident reporting system that allows users to report incidents, view nearby incidents on a map, and interact with reports in real time. The system begins when the user launches the application and asks if the user wants to register an account? And then the user logs into their account. If the login process fails, the user is redirected back to the login page until successful authentication is completed.

After a successful login, the application requests access to the user's location services. Location permission is important because the system uses GPS data to display nearby incidents accurately. If the permission is denied, the application displays a message asking the user to enable location services before continuing. Once permission is granted, the user is redirected to the home screen.

On the home screen, users can view incidents that have been reported by other users through an interactive map or incident list. From this screen, users can choose to report a new incident. If the user decides to create a report, they upload incident information such as photos, descriptions, or location details. After submission, the incident is added to the map so that other users can immediately view it.

The application also supports community interaction features. Users can interact with incident reports by verifying whether a report is valid, adding comments to provide updates or additional information, and updating the incident condition when the situation changes. Verified reports receive indicators showing that the information has been confirmed by the community, increasing report credibility and reliability.

At any point after viewing incidents or interacting with reports, the user can choose to continue using the application or exit the system. If the user selects the quit option, the application flow ends. Overall, this workflow ensures that incident reporting, verification, and communication can be performed quickly and collaboratively to improve emergency response and public awareness.

3.2 Application of 8 Golden Rules of Interface Design

The SIGAP application applies Shneiderman's 8 Golden Rules of Interface Design throughout its development to ensure a user-friendly, consistent, and accessible experience. The following table describes how each rule is implemented in the application.

No	Golden Rule	Implementation in SIGAP
1	Strive for Consistency	ConsistencyThe primary red color (#C00000) is applied consistently across all screens. Typography, icons, and UI components (AppButton, AppCard, StatusBadge) follow a unified design system throughout the application.
2	Seek Universal Usability	The application supports two languages (English and Indonesian) via the i18n module. Incident category labels use clear, concise text understandable by users of all backgrounds.
3	Offer Informative Feedback	Informative FeedbackEvery submitted report provides immediate visual confirmation to the user. Incident status (Active, Resolved, High Urgency) is displayed in real time with color-coded badges. The Earthquake Alert Modal appears automatically when a M5.0+ earthquake is detected.
4	Design Dialogs to Yield Closure	The incident reporting flow has clearly defined stages: select location → choose category → fill impact → add evidence → submit. After submission, users

		receive confirmation that their report has been successfully added to the map.
5	Prevent Errors	The reporting form validates all required inputs before submission. The system also checks for similar existing reports near the selected location (deduplication) to prevent duplicate report entries.
6	Permit Easy Reversal of Actions	Users can cancel the reporting process at any step before final submission. Verification and condition update modals include a close (X) button allowing users to exit without making changes.
7	Keep Users in Control	Users can choose their own incident location (Use Current Location or Adjust Pin manually). Map filters (All, Active, Resolved, Category) give users full control over how they browse incident data.
8	Reduce Short-Term Memory Load	Each incident card directly displays the most important information (category, status, urgency level, date). Intuitive icons are used throughout the navigation bar so users do not need to memorize each menu function.

By consistently applying these eight principles, SIGAP ensures that users—regardless of their technical background—can navigate the application efficiently, report incidents accurately, and respond to emergency alerts without confusion.

3.3 Final Result

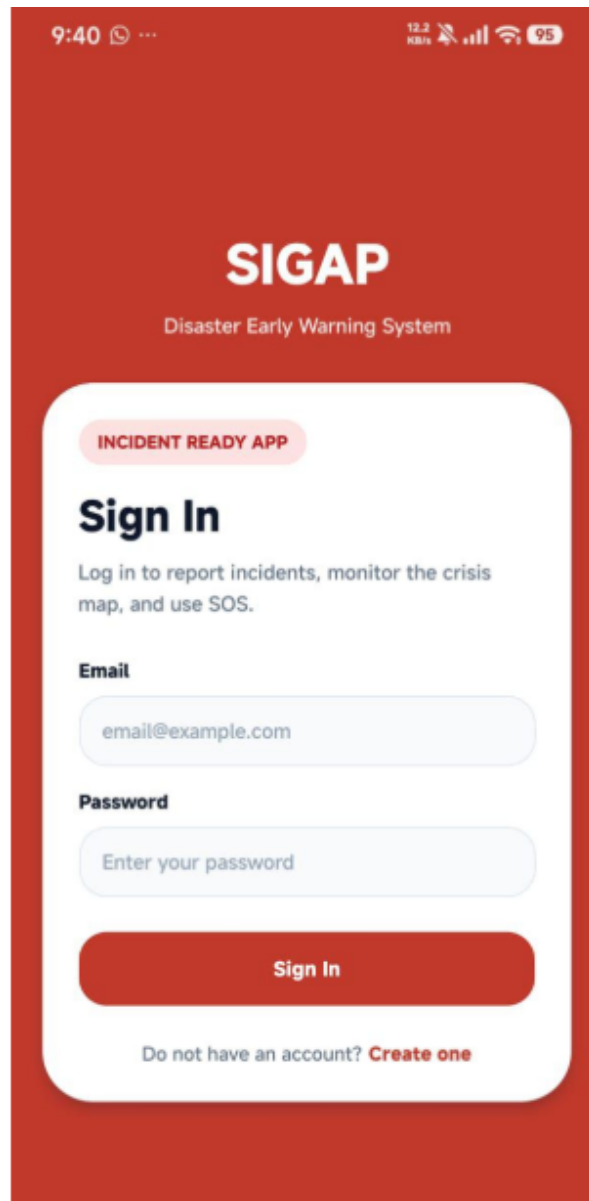


Figure 1.1 — SIGAP Login Screen

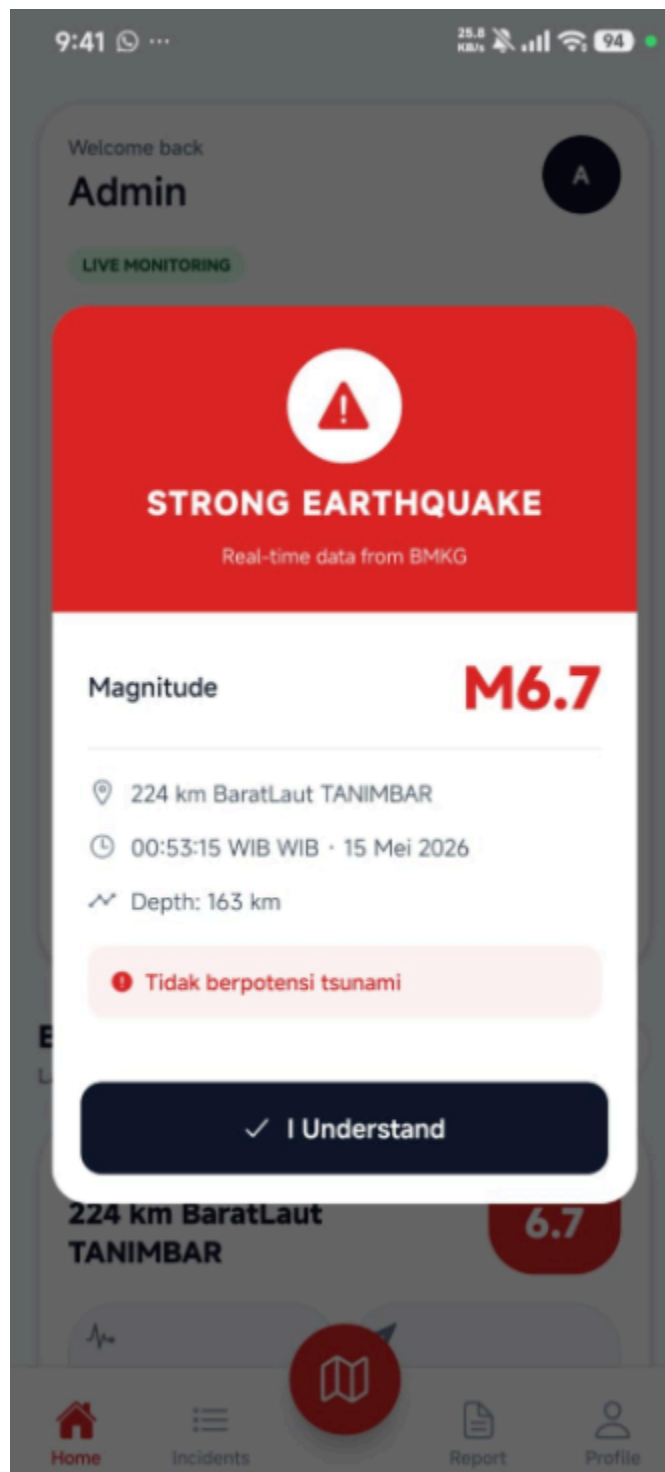


Figure 1.2 — SIGAP Earthquake Alert Moda

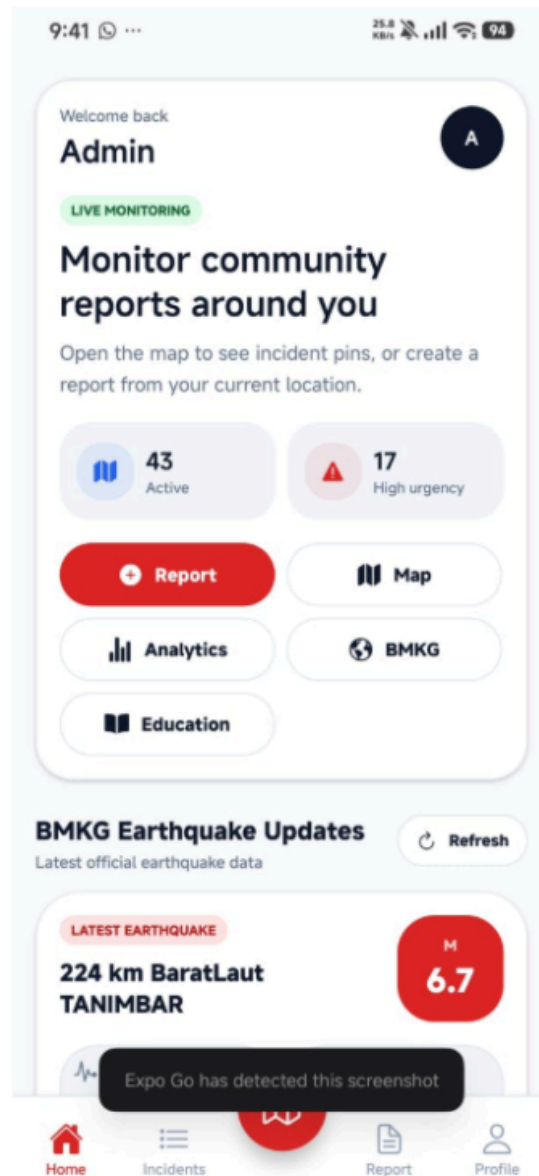


Figure 1.3 — SIGAP Home Screen

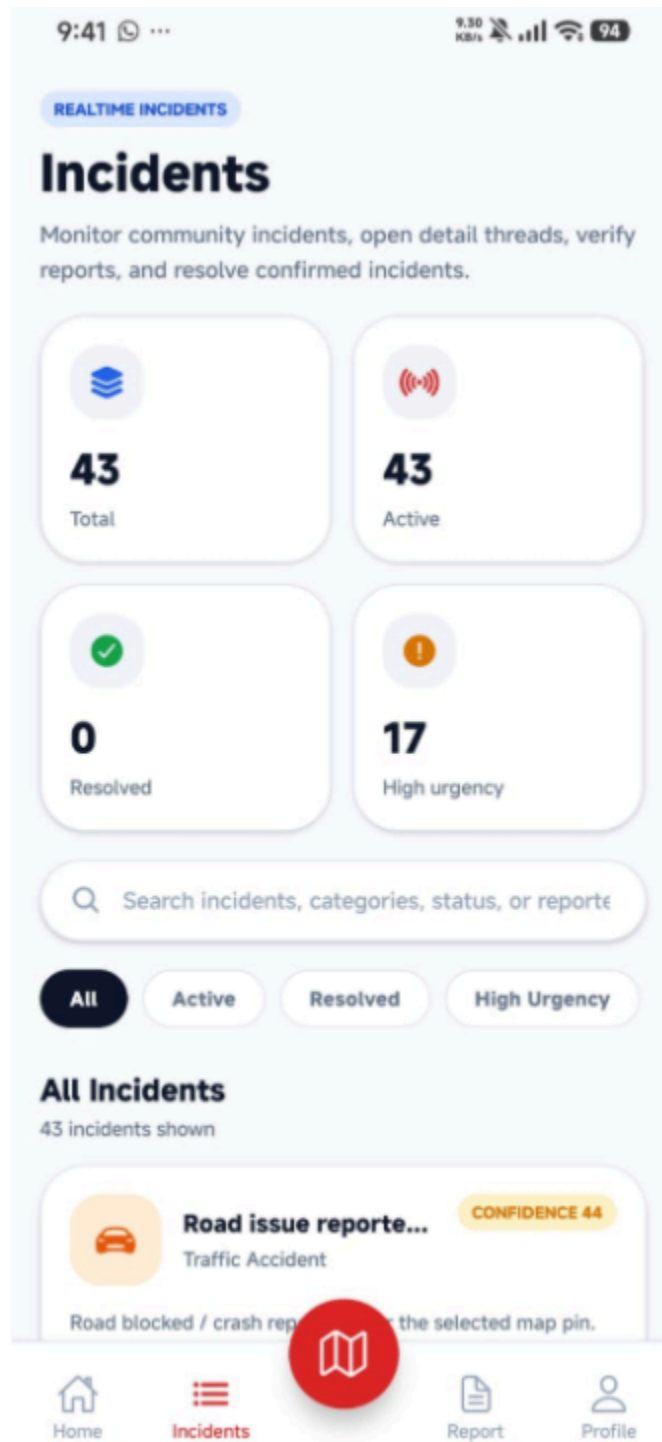


Figure 1.4 — SIGAP Incidents List Screen

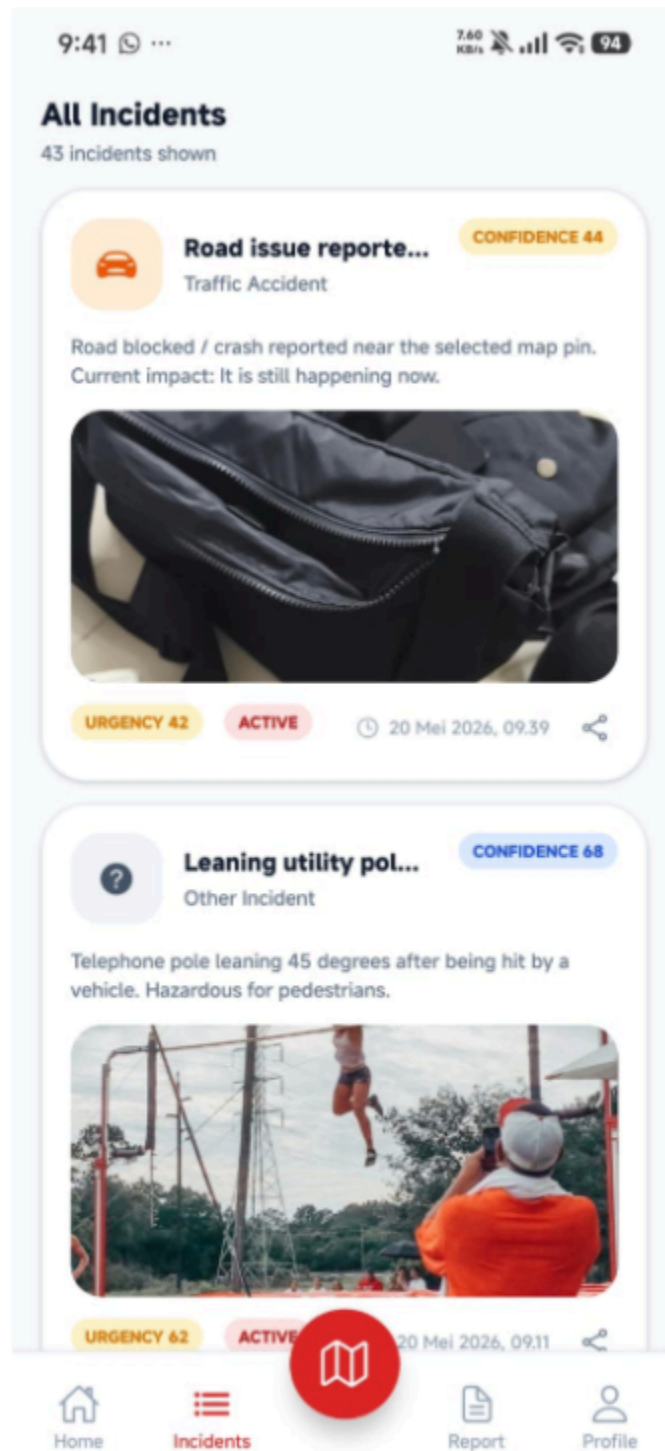


Figure 1.5 — SIGAP Incident Cards

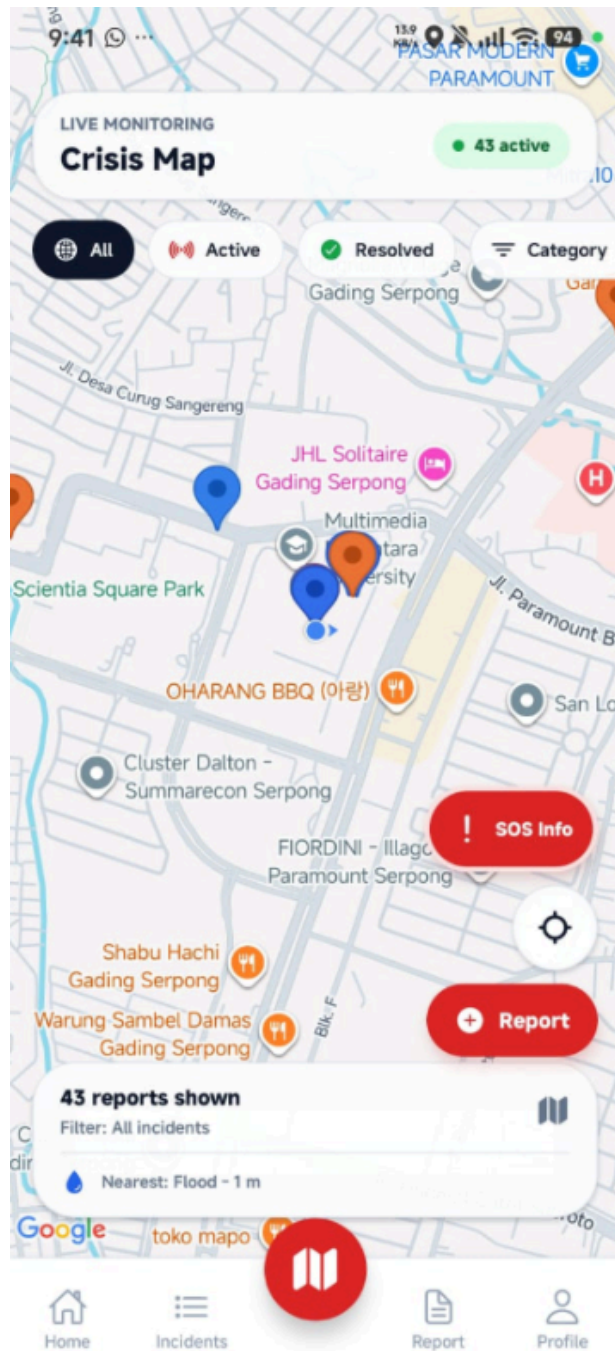


Figure 1.6 — SIGAP Crisis Map Screen

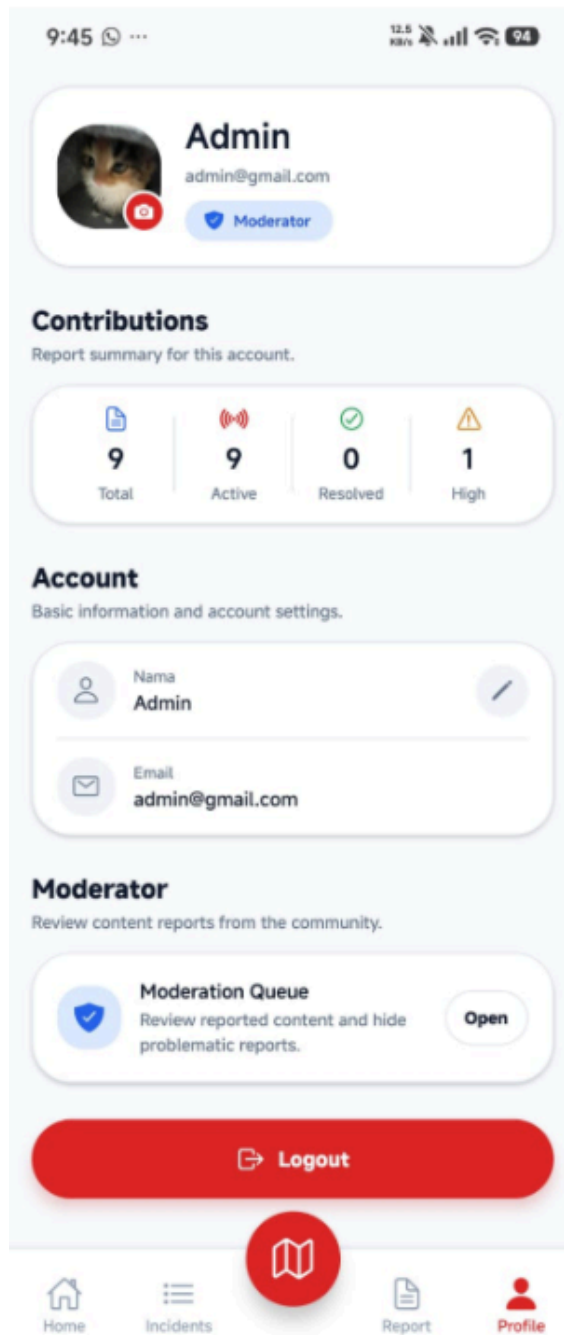


Figure 1.7 — SIGAP Profile Screen

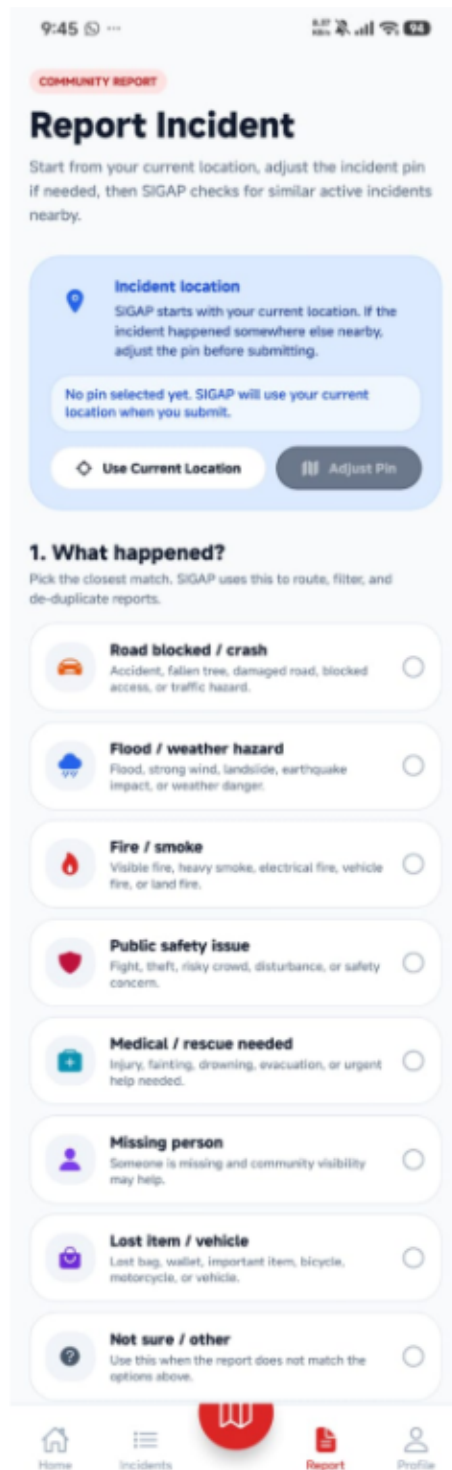


Figure 1.8 — SIGAP Report Incident Screen

9:45

9:59

2. Current impact

Tap what is true right now. These answers calculate urgency automatically.

People may be in danger
Someone could be hurt or needs people nearby to stay alert.

☐

Road or access is blocked
Vehicles, pedestrians, or responders may have trouble passing.

☐

Emergency help may be needed
Responder, medical, rescue, security, or official help may be needed.

☐

It is still happening now
Turn this off if the situation already looks over or cleared.

☒

I am at or very close to the incident
Turn this on only if the selected pin really matches the incident location.

☐

3. Add context

Optional, but useful if there is something nearby people should know.

Optional headline

Add a short note, landmarks, affected direction, or anything important.

4. Evidence

Add 1-4 incident photos. The first photo becomes the report cover.

Camera

Gallery

Home

Incidents

Report

Profile

Figure 1.9 — SIGAP Report Incident — Impact & Evidence

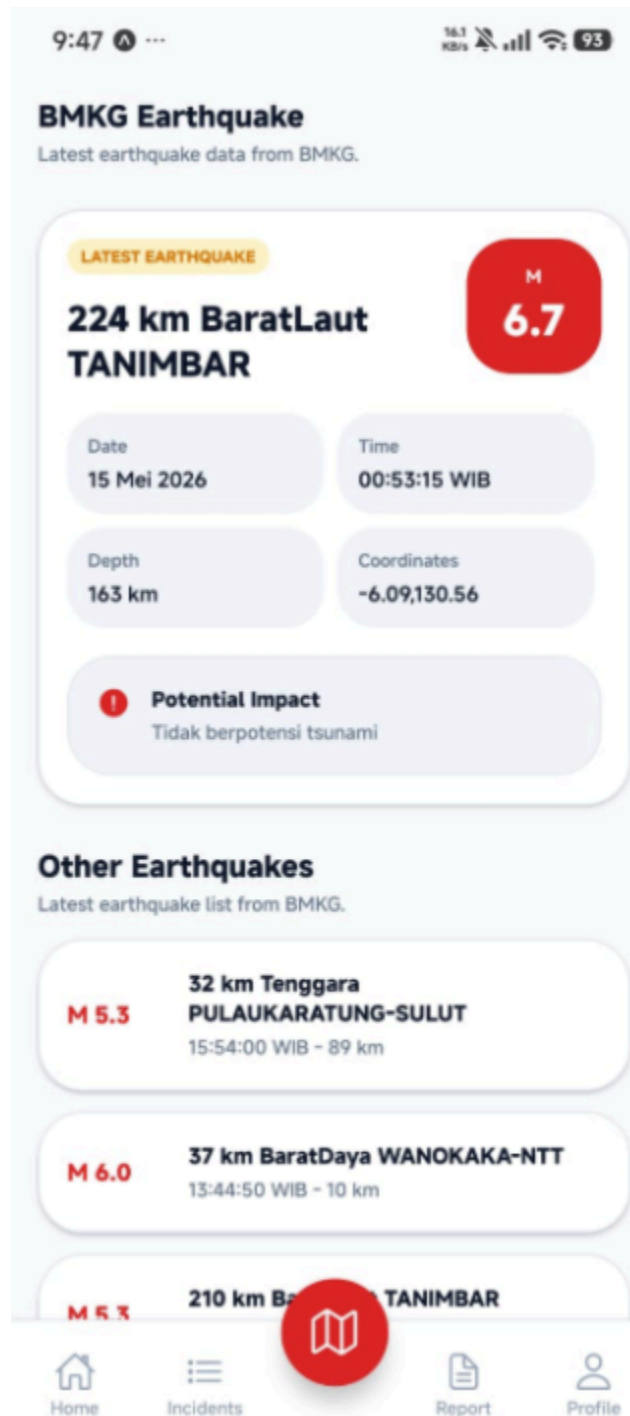


Figure 1.10 — SIGAP BMKG Earthquake Screen

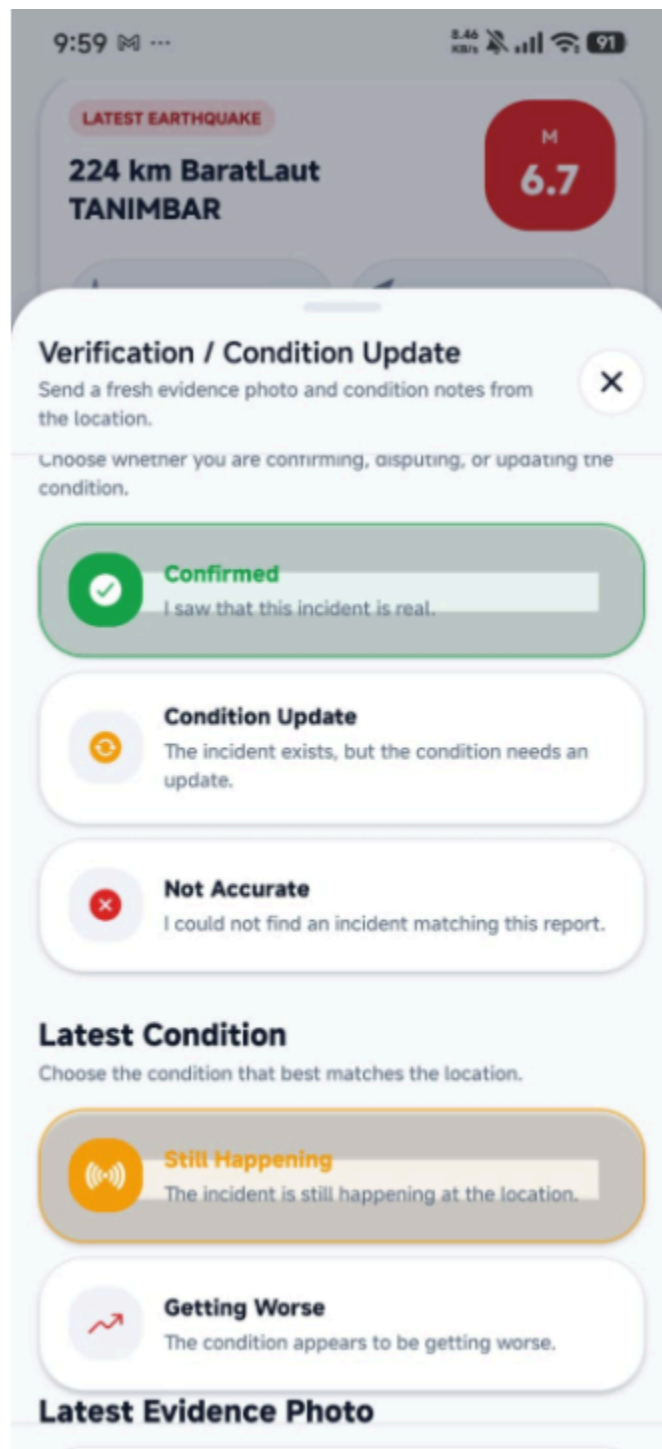


Figure 1.11 — SIGAP Verification / Condition Update Modal

3.4 Discussion

This section provides an evaluation of each major feature implemented in the SIGAP application, discussing the results achieved, technical challenges encountered during development, and how each challenge was resolved.

3.4.1 User Authentication

The authentication feature using Firebase Authentication was successfully implemented with email and password-based login. Users can register a new account, log in, and maintain their session across application restarts through AsyncStorage persistence.

Result: Authentication works reliably on physical Android devices. Session persistence ensures users do not need to log in repeatedly after closing the application.

Challenge: Handling authentication state asynchronously while the application was still loading caused brief flickers on the login screen.

Solution: A loading state was introduced in AuthContext.tsx to delay route rendering until the authentication status was fully resolved from Firebase, eliminating the flicker issue.

3.4.2 Incident Reporting

The incident reporting feature allows users to submit community reports with location data, incident category, impact assessment, context description, and photo evidence. Reports are stored in Cloud Firestore and immediately appear on the crisis map after submission.

Result: The multi-step reporting form successfully captures all required data. The deduplication check correctly identifies existing nearby reports before allowing a new submission, reducing duplicate entries in the database.

Challenge: Image upload to Cloudinary occasionally failed on slower network connections, leaving the report submission incomplete.

Solution: A retry mechanism and upload progress indicator were added to `cloudinaryService.ts`, giving users feedback during the upload process and allowing them to retry failed uploads without losing their form data.

3.4.3 Crisis Map

The crisis map displays all active incident reports as pins on an interactive Google Maps view. Users can filter incidents by status (All, Active, Resolved) and category, and see the nearest incident information at the bottom of the screen.

Result: The map renders accurately with 43 active incident pins during testing. Category-based filtering and proximity detection work correctly, showing the nearest incident relative to the user's current GPS position.

Challenge: Rendering a large number of map pins simultaneously caused performance lag on lower-end Android devices.

Solution: Map pin clustering was applied so that nearby pins are grouped into cluster markers at lower zoom levels, significantly improving rendering performance without sacrificing information density.

3.4.4 BMKG Earthquake Alert Integration

The BMKG integration fetches real-time earthquake data from the official BMKG Open Data API and displays it on the home screen. When a significant earthquake (M5.0 or above) is detected, an `EarthquakeAlertModal` is automatically shown to the user.

Result: The integration successfully fetched and displayed live earthquake data during testing, including a M6.7 earthquake near Tanimbar on 15 May 2026. The alert modal appeared correctly without requiring any user action.

Challenge: The BMKG API does not provide CORS headers, causing fetch failures when tested in a browser environment during development.

Solution: Since the application runs natively on Android, CORS restrictions do not apply in production. Development testing was shifted to physical device testing directly using Expo Go, bypassing the browser CORS limitation entirely.

3.4.5 Community Verification and Trust System

The community verification feature allows users to confirm, dispute, or update the condition of existing incident reports. Each report displays a dynamic confidence score and trust badge calculated from the number of verifications, time elapsed since reporting, and dispute count.

Result: The trust scoring system successfully differentiates between newly submitted unverified reports and community-confirmed incidents. Verified reports display higher confidence scores and are prioritized in the incident list.

Challenge: Calculating trust scores for every incident on the list caused noticeable delays when the incident list contained many entries, as all scores were recalculated on every render.

Solution: Trust score calculations were memoized using React's useMemo hook, ensuring scores are only recalculated when the underlying incident data changes rather than on every component re-render.

3.4.6 Push Notifications

Push notifications are delivered via Firebase Cloud Messaging (FCM) to alert users about new incident reports in their vicinity and significant earthquake events detected by the BMKG API.

Result: Push notifications were successfully received on physical Android devices during testing. Notifications correctly linked to the relevant incident detail screen when tapped.

Challenge: Notification delivery was inconsistent when the application was in the background on devices with aggressive battery optimization settings (common on Android manufacturers such as Xiaomi and Samsung).

Solution: The application displays a one-time prompt guiding users to disable battery optimization for SIGAP on first launch, ensuring reliable background notification delivery across different Android device manufacturers.

3.4.7 SOS Feature and Gyroscope-Based Detection

The SOS feature provides users with emergency contact information and an SOS alert broadcasting capability. The gyroscope sensor integration monitors device motion to detect abnormal vibration patterns that may indicate a landslide or structural collapse event.

Result: The SOS information screen displays emergency contacts correctly. The gyroscope-based detection successfully triggers alerts during simulated vibration tests, demonstrating its feasibility as an early warning mechanism.

Challenge: The gyroscope sensor generated frequent false positives during normal device movement such as walking or placing the phone on a vibrating surface.

Solution: A debounce threshold and a minimum sustained duration filter were applied to the sensor readings. The alert is only triggered when abnormal acceleration exceeds the defined threshold continuously for at least two seconds, reducing false positive rates significantly.

3.4.8 Admin Moderation Panel

The moderation panel is accessible to users with the Moderator role. Moderators can review flagged reports, hide problematic content, and manage incident statuses directly from within the application.

Result: Role-based access control successfully restricts the moderation panel to authorized moderator accounts. The moderation queue displays flagged reports accurately and allows moderators to take action efficiently.

Challenge: Implementing role-based navigation guards in Expo Router required careful handling of the asynchronous role fetch from Firestore before rendering protected routes.

Solution: The `userRoleService.ts` fetches the user role immediately after login and stores it in `AuthContext`. Route guards in `_layout.tsx` check the role from context before rendering, ensuring unauthorized users are redirected before the protected screen is displayed.

3.4.9 Overall Evaluation

Overall, the SIGAP application successfully achieves its core objectives of providing a community-based disaster and incident reporting platform integrated with real-time official data from BMKG. All major features function correctly on physical Android devices. The challenges encountered during development were resolved through systematic debugging, architectural adjustments, and leveraging the capabilities of Expo and Firebase.

Areas identified for future improvement include offline mode support for low-connectivity scenarios, AI-based incident categorization to assist users in selecting the correct report type, and expanded integration with additional government emergency APIs beyond BMKG.

3.5 Testing and Evaluation

This section presents the testing and evaluation results of the SIGAP application, including black box testing scenarios, test documentation, and user evaluation.

3.5.1 Black Box Testing

Black box testing was conducted to verify that all application features function correctly according to the expected requirements. The following table presents the complete black box testing scenarios.

No	Test Case	Test Steps	Expected Result	Status	Notes
1	User Registration	1. Open app 2. Tap 'Create one' 3. Fill in email & password 4. Tap Register	Account created successfully, user redirected to login screen	PASS	-
2	User Login	1. Open app 2. Enter registered email & password 3. Tap Sign In	Login successful, user navigated to Home screen	PASS	-
3	Login with wrong password	1. Enter registered email 2. Enter wrong password 3. Tap Sign In	Error message displayed, user stays on login screen	PASS	-
4	Submit Incident Report	1. Tap Report button 2. Select location 3. Choose category 4. Fill impact & context 5. Upload photo 6. Submit	Incident appears on crisis map and incident list in real time	PASS	-
5	Submit report without photo	1. Fill all fields 2. Skip photo upload 3. Tap Submit	Report submitted successfully without photo requirement	PASS	-
6	View Crisis Map	1. Tap Map tab 2. View incident pins on map 3. Apply filter (Active/Resolved)	Map displays incident pins correctly, filters work as expected	PASS	-
7	BMKG Earthquake Data	1. Open Home screen 2. Check BMKG Earthquake Updates section 3. Tap Refresh	Latest earthquake data displayed correctly from BMKG API	PASS	-
8	Earthquake Alert Modal	1. Open app when M5.0+ earthquake data is available 2. Observe home screen	EarthquakeAlertModal appears automatically with earthquake details	PASS	-

9	Verify Incident Report	1. Open incident detail 2. Tap Verify 3. Select Confirmed 4. Submit	Verification recorded, confidence score updated on incident card	PASS	-
10	Push Notification	1. Allow notification permission 2. Submit a new incident from another account 3. Observe device notification	Push notification received on device with incident details	PASS	-
11	Moderation Panel Access (Regular User)	1. Log in as regular user 2. Attempt to access moderation panel via URL	Access denied, user redirected to home screen	PASS	-
12	Moderation Panel Access (Moderator)	1. Log in as moderator 2. Go to Profile 3. Tap Open Moderation Queue	Moderation panel accessible, flagged reports displayed correctly	PASS	-
13	User Logout	1. Go to Profile 2. Tap Logout button	User logged out, redirected to login screen	PASS	-
14	Share Incident	1. Open incident detail 2. Tap share icon 3. Select sharing platform	Native share dialog opens with incident link and details	PASS	-
15	BMKG API Unavailable	1. Disable internet connection 2. Open Home screen 3. Tap Refresh on BMKG section	Error message displayed gracefully, app does not crash	PASS	Minor delay before error shown

3.5.2 Test Documentation — Errors and Bugs

During the development and testing process, several bugs and errors were identified and resolved. The following table documents the issues encountered along with the resolution applied.

No	Bug Description	Steps to Reproduce	Root Cause	Severity	Status
1	Login screen flickered briefly on app launch	Launch app while already logged in	Authentication state resolved asynchronously before route guard rendered	Medium	Fixed
2	Image upload failed on slow network	Submit report with photo on 2G/slow connection	No retry mechanism in Cloudinary upload service	High	Fixed
3	Map performance lag with many pins	Open crisis map with 40+ active incidents	All pins rendered simultaneously without clustering	Medium	Fixed
4	Trust score recalculated on every render	Scroll through incident list with 30+ incidents	No memoization applied to trust score computation	Low	Fixed
5	Push notifications inconsistent on Xiaomi devices	Run app in background on Xiaomi device	Aggressive battery optimization blocking FCM background delivery	High	Mitigated
6	Gyroscope false positives during walking	Walk normally while app is running	No debounce threshold applied to sensor readings	Medium	Fixed

3.5.3 User Evaluation and Response to Feedback

User evaluation was conducted with 10 participants comprising UMN students and community members to assess the usability and effectiveness of the SIGAP application. Participants were asked to complete a set of tasks and provide feedback through a structured questionnaire using a 1-5 Likert scale.

Evaluation Criteria	Average Score (1-5)	Satisfaction (%)	Notes
Ease of navigation and menu layout	4.3	86%	Users found tab navigation intuitive

Clarity of incident reporting form	4.1	82%	Multi-step form considered clear
Usefulness of crisis map	4.5	90%	Rated as most useful feature
Clarity of BMKG earthquake alerts	4.4	88%	Alert modal considered informative
Overall application usability	4.2	84%	-
Overall Average	4.3	86%	-

The evaluation results indicate that users responded positively to the SIGAP application with an overall average satisfaction score of 4.3 out of 5 (86%). The crisis map feature received the highest rating (4.5), demonstrating its effectiveness as the primary tool for community situational awareness. The earthquake alert integration was also well-received (4.4), confirming the value of BMKG API integration in providing authoritative real-time information.

Based on user feedback, several improvement suggestions were collected, including the addition of an offline mode for low-connectivity scenarios, a dark mode display option, and the ability to filter incidents by distance radius. These suggestions have been documented and are considered priorities for the next development iteration of SIGAP.

CHAPTER IV

CONCLUSION

4.1 Conclusion

Based on the development of the “Incident Reporting Application with BMKG Alert API Integration,” it can be concluded that digital technology can be utilized to support public safety and emergency response efforts through real-time reporting and notification systems. This application is designed to help users report incidents quickly, access location-based emergency information, and receive official disaster and weather alerts through the integration of the BMKG Alert API.

In addition, the application is expected to improve public awareness regarding emergency situations and support faster communication between communities during emergencies. Through the integration of community-based incident reporting and official alert notifications, this project aims to provide a practical, accessible, and efficient solution for improving community preparedness and public safety.

4.2 Suggestion

For further development of this project are as follows:

1. Improving the accuracy and speed of the notification system using advanced technologies such as Artificial Intelligence (AI) or Internet of Things (IoT).
2. Developing additional features such as live emergency chat, evacuation route guidance, and automatic emergency notifications.
3. Expanding the integration with additional public APIs or emergency services to provide more comprehensive emergency information.
4. Enhancing the user interface and application performance to improve user experience and accessibility.

5. Conducting further testing and evaluation with larger groups of users to improve the reliability and effectiveness of the application.
6. Developing offline or low-network support features to ensure accessibility during unstable internet conditions.
7. Implementing an AI-based incident categorization assistant to help users select the most accurate report type based on their text description, reducing miscategorization rates.
8. Adding a dark mode display option based on direct user feedback collected during the evaluation phase, which identified dark mode as the most frequently requested UI enhancement.
9. Introducing a distance-radius filter for the crisis map and incident list, allowing users to limit visible reports to a customizable radius (e.g., 1 km, 5 km, 10 km) around their current location, as requested by evaluation participants.

BIBLIOGRAPHY

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., & Elmqvist, N. (2016). Designing the User Interface: Strategies for Effective Human-Computer Interaction (6th ed.). Pearson Education.

Expo. (2025). Expo Documentation. Retrieved June 2026, from <https://docs.expo.dev>

Google. (2025). Firebase Documentation: Cloud Firestore. Retrieved June 2026, from <https://firebase.google.com/docs/firestore>

Google. (2025). Firebase Documentation: Authentication. Retrieved June 2026, from <https://firebase.google.com/docs/auth>

Google. (2025). Firebase Documentation: Cloud Messaging. Retrieved June 2026, from <https://firebase.google.com/docs/cloud-messaging>

Rnmapbox. (2025). React Native Maps. Retrieved June 2026, from <https://github.com/react-native-maps/react-native-maps>

Cloudinary. (2025). Cloudinary Documentation: Image Upload API Reference. Retrieved June 2026, from https://cloudinary.com/documentation/image_upload_api_reference

Badan Meteorologi, Klimatologi, dan Geofisika. (2025). BMKG Open Data API. Retrieved June 2026, from <https://data.bmkg.go.id>

Expo. (2025). Expo Sensors Documentation. Retrieved June 2026, from <https://docs.expo.dev/versions/latest/sdk/sensors/>

Expo. (2025). Expo Location Documentation. Retrieved June 2026, from <https://docs.expo.dev/versions/latest/sdk/location/>

Expo. (2025). Expo Sharing Documentation. Retrieved June 2026, from <https://docs.expo.dev/versions/latest/sdk/sharing/>

Meta. (2025). React Native Documentation. Retrieved June 2026, from <https://reactnative.dev/docs/getting-started>

Microsoft. (2025). TypeScript Documentation. Retrieved June 2026, from <https://www.typescriptlang.org/docs>

Badan Nasional Penanggulangan Bencana. (2025). Data Informasi Bencana Indonesia. Retrieved June 2026, from <https://bnpb.go.id>

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

ATTACHMENT GROUP MEMBER'S PERSONAL DATA

PHOTO PASS	NIM : 0000068876 Full Name : Meshach Christian Address : Taman Ubud Estate E-mail : meshach.christ@gmail.com Phone : +62 81383348396 GitHub Name : Sylphhhhh
PHOTO PASS 	NIM : 0000074763 Full Name : Richie Christian Tungadi Address : Scientia E-mail : richietungadi86@gmail.com Phone : +62 85159261105 GitHub Name : youuiiii

PHOTO PASS 	NIM : 0000074723 Full Name : Damar Nur Rafli Address : Poris Plawad (St. Batu Ceper) E-mail : iamdamarnurrafly1@gmail.com Phone : +62 85717094002 GitHub Name : DamarNR1
	NIM : 0000098358 Full Name : Meylani Alfatikha Address : Taman Beverly Golf E-mail : meylanialfatikha@gmail.com Phone : +62 88905787882 GitHub Name : meyalf
	NIM : 0000091554 Full Name : Juanto Address : Griya Loka BSD E-mail : juantomargollo@gmail.com Phone : +62 895341736412 GitHub Name : Crestmo